

02

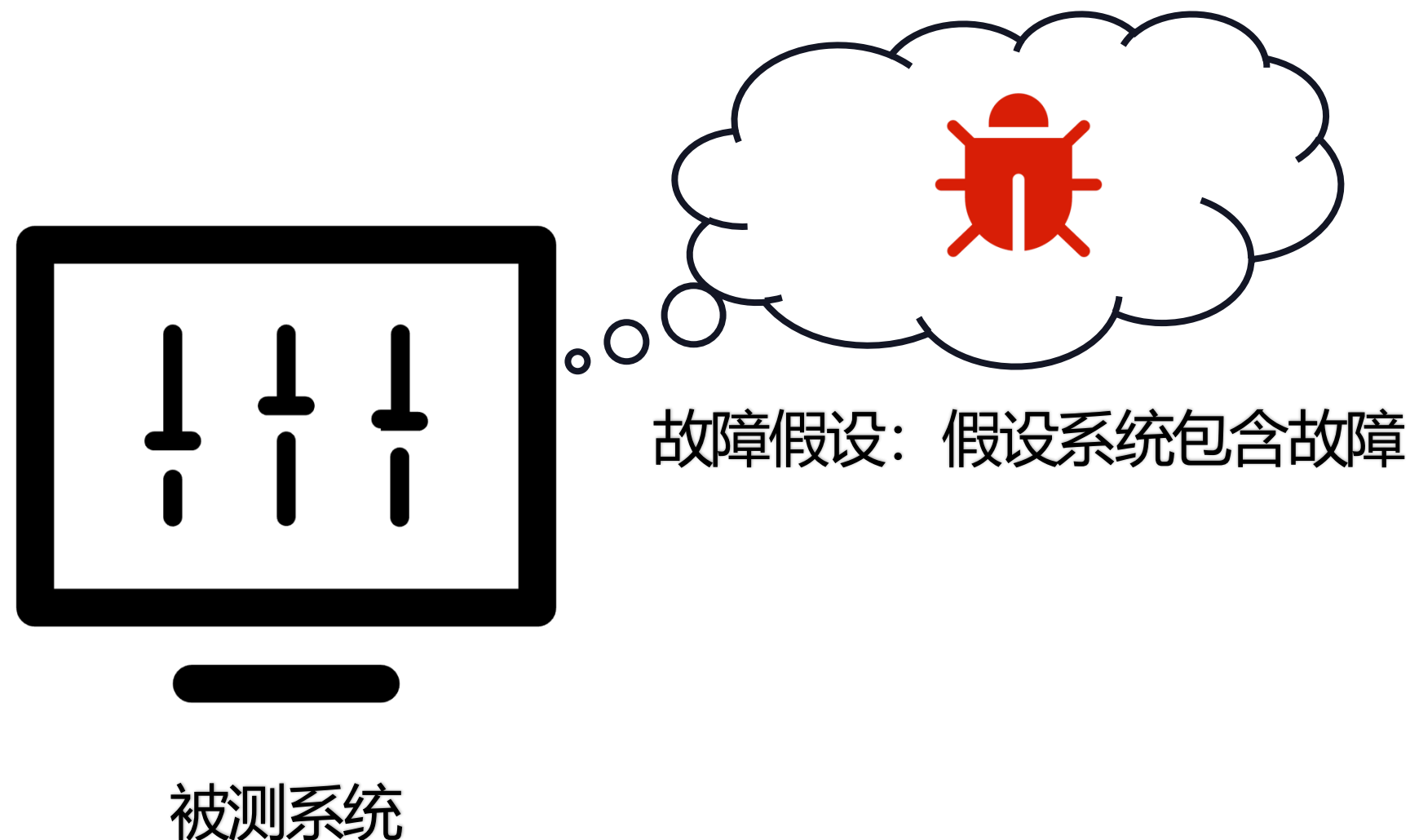
故障假设开发者测试



故障假设开发者测试概述

什么是故障假设测试？

故障假设是开发者测试中的一种重要方法，帮助识别应用程序中的问题和漏洞。开发者根据应用程序的特点和使用情况，提出可能的故障假设，如程序崩溃或逻辑错误，并设计测试来验证这些假设。这种方法可以与其他测试方法（如路径分析和代码可达性）结合使用，以更全面地理解应用程序的行为，提高测试效率。



● 黑盒与白盒测试的边界值分析对比

从输入约束到执行路径的关键差异

维度	黑盒测试	白盒测试
输入变量假设	变量独立，仅关注单个变量边界	多约束条件共同作用
边界输入生成逻辑	简单边界值（如min, max）	复杂子空间的多重边界组合
测试值选择	需非边界值，用例数量多	聚焦路径边界值，用例更精准
主要挑战	输入域的完整性	执行边界的定义与覆盖

白盒边界值分析的挑战

问题1：边界值的“距离”定义

- 变量类型影响：整数与浮点变量的边界距离计算方式不同（如整数需考虑“多1/少1”错误）。
- 多维约束复杂性：约束 $2x + 3y < 18$ 存在多个子空间边界（如 (2,4) 和 (4,3)，差值分别为 2 和 1）。

问题2：路径条件与边界值的关联

- 路径条件示例： $(a > 0) \vee (b > 0)$ 。
- 边界值矛盾：输入 (1,5) 接近 $a > 0$ 的边界，但并非真正的边界值。
- 正确边界值：(1, -1)（使路径条件从真变为假的最小变化点）。

物理边界与确定边界

物理边界与确定边界概述

- 物理边界：输入稍作改变（如微小扰动）会导致路径条件从真变假。示例：约束 $a > 0 \rightarrow$ 边界值 $a=1$ 。
- 确定边界：输入使某个谓词直接决定路径条件的评估结果（改变该谓词的值会改变路径条件）。示例：约束 $(a > 0) \vee (b > 0) \rightarrow$ 边界值 $(1, -1)$ （仅由 $a > 0$ 决定）。

路径边界确定条件定义：

定义 6.2: 边界确定条件

在给定输入情况下，谓词 c_i 确定路径 PC 的条件定义为：

$$PC \oplus PC_{c_i, \neg c_i} \quad (6.6)$$

测试生成技术的优化

传统方法的缺陷

直接对每个条件变异会导致组合爆炸（测试用例数量过大）。

优化策略

- 组合测试生成：

对每条路径建立组合模型，生成兼容的边界条件集合（覆盖多个边界的测试用例）。

示例：一条路径的多个边界条件可通过一个测试用例同时覆盖。

- 约束求解器：将组合约束转换为具体输入值，确保测试效率。

- 保留覆盖标准：在减少测试数量的同时，仍满足路径覆盖要求。

白盒测试中的边界值分析实例

场景：在白盒测试中，生成一条到达代码第6行的执行路径，路径条件为 $PC = w \wedge x > 1 \wedge x < 10 \wedge y \geq z$ 。

目标：通过边界值分析生成测试用例，覆盖路径条件的边界情况，同时避免组合爆炸问题。

表 6.8: 路径确定条件分析

c	$p_{c'}$	值	$PC_{c,c'}$
w	NA	NA	NA
$x > 1$	$p_{x>2}$	$\{0, 1\}$	$w \wedge x > 2 \wedge x < 10 \wedge y \geq z$
	$p_{x>0}$	$\{0, 1\}$	$w \wedge x > 0 \wedge x < 10 \wedge y \geq z$
$x < 10$	$p_{x<11}$	$\{0, 1\}$	$w \wedge x > 1 \wedge x < 11 \wedge y \geq z$
	$p_{x<9}$	$\{0, 1\}$	$w \wedge x > 1 \wedge x < 9 \wedge y \geq z$
$y \geq z$	$p_{y\geq z+1}$	$\{0, 1\}$	$w \wedge x > 1 \wedge x < 10 \wedge y \geq z + 1$
	$p_{y\geq z-1}$	$\{0, 1\}$	$w \wedge x > 1 \wedge x < 10 \wedge y \geq z - 1$

变异测试概述

什么是变异测试？

变异测试是故障驱动的治疗方法，通过生成与源程序差异极小的变异体（模拟缺陷），对比测试结果差异来评估测试集的缺陷检测能力。

变异测试核心作用

暴露测试不足，优化测试数据集

主动模拟潜在故障场景，提升测试覆盖率

测试差分器

测试差分器 $d : T \times P \times P \rightarrow \{0, 1\}$

$$d(t, P, P') = \begin{cases} 1 & t \text{ 对于 } P \text{ 和 } P' \text{ 行为不同} \\ 0, & \text{否则} \end{cases} \quad (6.12)$$

这是一个评估所有测试用例 $t \in T$ 和程序 $P, P' \in P$ 行为差异的函数。为简化讨论，我们用 M 表示 P 和 P' 的行为差异集合，而 $d(M)$ 表示满足这种行为差异的约束条件或所有可能输入的集合。

变异算子

变异算子概述

变异算子是变异测试中的核心概念，用于指导程序变异。通过变异算子对原始程序进行微小修改，生成变异体，用于评估测试用例集质量和测试过程有效性。

不同程序类型适用的变异算子不同

- 面向过程程序可用运算符、数值、方法返回值变异等算子；
- 面向对象程序除这些算子外，还需针对继承、多态、重载等特性设计新算子，以保证特征覆盖完整。

面向过程程序的变异算子简要总表

表 6.9: 面向过程程序的变异算子

变异算子	描述
条件变异	对关系运算符 “<”、“<=”、“>”、“>=” 进行替换，如将 “<” 替换为 “<=”
数学变异	对自增运算符 “++” 或自减运算符 “-” 进行替换，如将 “++” 替换为 “-”
二元变异	对与数值运算的二元算术运算符进行替换，如将 “+” 替换为 “-”
否定变异	将程序中的条件运算符替换为相反运算符，如将 “==” 替换为 “!=”
数值变异	对程序中整数类型、浮点数类型的变量取相反数，如将 “i” 替换为 “-i”
返回值变异	删除程序中返回值类型为 void 的方法；对程序中方法的返回值进行修改，如将 “true” 修改为 “false” 等

PITest工具概述

PITest工具介绍

PITest 是一款由 Henry Coles 等人开发的 Java 变异测试工具。它配置方便，易于使用，能与多种 Java 开发工具（如 Ant、Maven、Gradle）及平台（如 Eclipse、IntelliJ）良好集成，可快速搭建变异测试环境。

PITest工具优势

- 高效变异测试
- 生成测试报告
- 评估测试质量

常用的变异算子及其应用

面向对象程序中，常用的变异算子可以分为几大类：

- 改变方法调用：可替换为其他或空方法调用，或删除调用，测试不同输入及缺少特定功能时程序的表现。
 - 改变方法参数：可改变参数类型、顺序或删除参数，测试对不同数据、组合及缺少信息的处理。
 - 改变方法体：可替换或删除方法体中的语句，测试不同算法及缺少功能时的正确性和鲁棒性。
 - 改变类继承：可改变或删除类的继承关系，测试不同继承层次及缺少功能时的表现。
 - 改变类成员：可改变字段或方法，或删除它们，测试对不同数据、算法及缺少信息的处理。
 - 改变访问权限：可改变类或成员的访问权限，测试不同权限下的正确性和鲁棒性，需谨慎使用。
- 这些变异算子通过微小改动评估程序表现，提升可靠性和稳定性。

企业界使用 PITest 等变异测试工具面临的挑战与必要性

面向对象程序中，常用的变异算子可以分为几大类：

- 规模和效率问题：在大规模代码库（如谷歌的代码库）上运行变异测试非常昂贵且耗时，现有的优化方法虽然可以减少需要分析的变异体数量，但仍然不足以使变异检测率的计算变得可行。
- 报告的可操作性问题：变异检测率的计算成本高，且难以向开发者提供具体的、可操作的反馈，无法有效指导测试工作。
- 非生产性变异体的挑战：大规模报告个体变异体对开发者来说具有挑战性，特别是因为存在许多非生产性的变异体（即对测试没有实际帮助的变异体）。

应对挑战的必要性

- 需要设计并实施不同于传统方法的变异测试方法，以应对规模和非生产性变异体带来的挑战，提高变异测试在企业环境中的可行性和实用性。